Requirements Gathering

Real-Time Supply Chain Monitoring Platform

Purpose and Approach

Requirements gathering defines the project goals, boundaries, user needs, and conditions for accepting the final system before implementation begins.

This document establishes the initial requirements for the Real-Time Supply Chain Monitoring Platform. It identifies stakeholders, defines the elicitation approach, records functional and non-functional requirements, prioritizes deliverables, and establishes validation and change-control procedures.

The requirements represent a proposed baseline. The team must confirm the data source, available fields, technical feasibility, and acceptance criteria before treating the baseline as final.

Stakeholders

Stakeholders are identified according to their interest in the project and their role in defining, developing, evaluating, or using the platform.

Stakeholder	Interest and Requirements
Supply-chain managers and operations users	View shipment statuses, identify delivery delays, inspect recent events, and monitor inventory levels across warehouses or distribution hubs.
Project supervisor and DEPI evaluator	Confirm the project scope, required deliverables, technical depth, data quality practices, Big Data implementation, testing evidence, and documentation.
Data provider or dataset owner	Clarify available fields, data format, update frequency, data permissions, source reliability, and known limitations.
Development team	Agree on the schema, data-cleaning rules, database design, pipeline behavior, tool availability, testing procedures, and delivery criteria.

Elicitation Plan

The team will use collaborative methods to identify business needs, investigate available data, and agree on measurable requirements before implementation.

Data Profiling and Schema Review

The team will inspect representative CSV or JSON records to identify missing values, duplicates, invalid timestamps, inconsistent status values, and relationships between shipment and event records.

The results will determine which requirements can be implemented using the available data.

Collaborative Requirements Workshop

The team will agree on the main business questions, shipment status definitions, delay calculation rules, inventory indicators, data model, and minimum dashboard features.

The project supervisor will be consulted to confirm the expected deliverables and whether simulated streaming, Kafka, and Airflow are required or optional.

Peer Review and Testing

Team members will review each other's code, SQL queries, transformation results, and dashboard metrics.

Each core component must have a defined owner and a separate reviewer. This approach supports shared understanding without requiring every member to implement the same component.

Business Requirements

The platform must help users answer operational questions using shipment and inventory data.

Shipment Tracking

Users need to inspect the latest known status of each shipment, including statuses such as Pending, In-Transit, and Delivered, where supported by the selected dataset.

The platform must retain enough event information to explain how the latest known status was determined.

Delivery Delay Analysis

Users need to identify shipments that arrive later than expected and determine the duration of the delay.

Where the required timestamps exist, delay duration will be calculated from the actual and expected delivery times. Shipments without sufficient timestamp information must not be assigned an invented delay value.

Inventory Monitoring

Users need to inspect inventory quantities across available warehouses or distribution hubs and identify changes over time.

This capability depends on the availability of inventory records and timestamps. If the selected dataset contains shipment records only, inventory monitoring will require another data source or clearly labeled sample data.

Operational Metrics

The initial dashboard should provide the following metrics where the required fields are available:

- Total number of shipments.
- Number of shipments by current status.
- Number of delayed shipments.
- On-time delivery rate for shipments with sufficient delivery information.
- Delay duration for eligible shipments.
- Inventory levels by warehouse or hub, when inventory data is available.

The team must document the calculation rules and denominator used for each metric.

Functional Requirements

The functional requirements describe the behavior that the platform must provide.

Requirement	Description and Acceptance Check
Data ingestion	The system must ingest the agreed CSV or JSON source using Python. A test file must load successfully, and the run summary must report the number of records processed.
Raw data preservation	The pipeline should preserve the original input or an equivalent raw-data copy so that cleaned records can be traced back to their source.

Requirement	Description and Acceptance Check
Data validation and cleaning	The system must validate required fields, data types, identifiers, timestamps, and allowed status values. It must detect duplicates, missing values, and invalid records according to documented rules.
Relational storage	The system must store the agreed structured data in Microsoft SQL Server. The team must demonstrate successful loading and retrieval through SQL queries.
Shipment status calculation	The system must determine the latest known shipment status from available event records. When events arrive out of chronological order, the result must follow the documented event-time rules rather than file order alone.
Delay calculation	The system must identify delayed shipments when the necessary expected and actual delivery timestamps are available. A test shipment with an actual delivery time after its expected delivery time must produce the correct delay duration.
Big Data processing	The pipeline must load the agreed batch dataset into HDFS, apply distributed transformations using PySpark, and write curated analytical outputs in Parquet format.
Data reconciliation	The team must compare SQL and Spark results using the same input scope and transformation rules. Row counts, schema compatibility, and selected business metrics must match where the outputs are intended to represent the same data.
Simulated event processing	The platform must support sequential replay of sample shipment events through a Python script or an optional Kafka producer/consumer prototype. Replaying an event must not create unintended duplicate records.
Dashboard visualization	The Streamlit dashboard must display shipment statuses, delay indicators, and available inventory metrics. Displayed values must be traceable to the corresponding SQL queries or validated Parquet outputs.
Workflow orchestration	Where feasible, Apache Airflow may schedule and monitor pipeline tasks. If it cannot be configured within the available environment and timeline, the core pipeline must remain executable without it.
Testing and documentation	The team must provide test cases, test results, setup instructions, the Data Dictionary, and documentation of known limitations.

Non-Functional Requirements

These requirements describe the quality, reliability, and maintainability expected from the platform.

Quality Attribute	Requirement and Verification
Repeatability	Repeated pipeline executions must not create unintended duplicate records for the same defined business key or event identifier. The team must verify this using repeated runs against the same input.

Quality Attribute	Requirement and Verification
Data accuracy	Dashboard totals and selected indicators must match independent SQL results calculated over the same dataset and under the same business rules.
Data traceability	The team must document source fields, schema definitions, transformation rules, invalid-record handling, and the relationship between source records and output metrics.
Reliability	Invalid records and failed pipeline tasks must produce actionable error information. The pipeline must not silently report a successful run when required processing fails.
Maintainability	Code must be organized into understandable ingestion, validation, transformation, loading, and dashboard modules. Configuration and schema changes must be documented.
Version control	Project code, schemas, configuration examples, and documentation must be maintained in a shared GitHub repository with a consistent folder structure.
Usability	Dashboard labels, units, filters, and metric definitions must be understandable without requiring users to write SQL queries.
Feasibility	The core MVP must run using tools and computing resources available to the team. Optional infrastructure must not prevent delivery of the core requirements.

Constraints and Assumptions

Project Constraints

- The project must follow the agreed four-week execution plan.
- Microsoft SQL Server is the proposed relational database and must be tested on the team's available environment.
- Streaming events generated by a simulator must be labeled "Simulated" and must not be represented as live operational data.
- Production-grade integration with physical IoT sensors or live carrier systems is outside the current MVP scope.
- The project must use data that the team is permitted to access and share.

Assumptions and Open Decisions

- A suitable sample dataset or simulated data source can be obtained without paid access.
- The final dataset, schema, and available inventory fields remain subject to source investigation.
- HDFS and PySpark must be tested in the available environment before the Big Data phase is finalized.
- Kafka and Airflow are optional unless the project supervisor confirms otherwise.
- The final processing-time target will be established after measuring the selected dataset and execution environment.

MoSCoW Prioritization

The requirements are prioritized according to business value, dependencies, feasibility, and the four-week timeline.

Priority	Scope	Target Phase
Must Have	Dataset selection, schema design, Data Dictionary, Python ingestion and cleaning, SQL Server storage, core status and delay logic, essential SQL queries, testing, and basic dashboard.	Weeks 1–4
Should Have	HDFS storage, PySpark transformations, Parquet outputs, SQL/Spark reconciliation, dashboard filters, and documented data-quality results.	Weeks 3–4
Could Have	Kafka-based event replay, Airflow DAG scheduling, additional metrics, and extended visualizations.	Weeks 3–4, subject to feasibility
Won't Have in the MVP	Production-grade IoT integration, guaranteed live carrier feeds, enterprise-scale infrastructure, and production alerting services.	Future extensions

Validation and Acceptance

Requirements must be reviewed by the team and project supervisor before implementation. Each core requirement must have a documented acceptance check and a corresponding test result.

Data Quality Validation

The test dataset must include normal records and relevant edge cases, such as:

- Missing or invalid timestamps.
- Duplicate event identifiers.
- Missing required fields.
- Unknown shipment references.
- Events arriving out of chronological order.
- Shipments delivered before their expected delivery time and shipments delivered after that time.

The team must document how each case is handled. Invalid records must be rejected, quarantined, or handled according to an agreed rule.

Pipeline Validation

The team must verify that the pipeline can process the agreed input, preserve source traceability, and execute repeatedly without unintended duplication.

Failures must be logged with enough information to identify the affected task and investigate the cause.

SQL and Spark Reconciliation

The SQL Server and PySpark results must be compared using the same input records and equivalent business rules.

The comparison must include:

- Input and output row counts.
- Required fields and data types.
- Duplicate and invalid-record handling.
- Shipment status and delay metrics.

Any expected differences must be explained and documented before the results are accepted.

Dashboard Validation

Each displayed metric must have a documented definition and an independent verification query or equivalent validated calculation.

The team must confirm that dashboard totals, filters, and shipment-level details correspond to the underlying data.

Acceptance Decision

A requirement is accepted when its documented acceptance check passes and the team can demonstrate the expected behavior.

Failed checks must be recorded, investigated, and retested after corrective changes.

Requirements Traceability and Change Control

Traceability

The team will maintain a shared requirements tracker linking each requirement to its priority, responsible component, implementation location, acceptance check, test result, and current status.

This tracker will help the team identify incomplete work and verify that each required feature has been implemented and tested.

Baseline Configuration

The shared GitHub repository will serve as the baseline for project code, schemas, configuration examples, and documentation.

Changes to the approved requirements or technical design must be recorded with their reason and expected impact.

Scope Management

The core MVP must take priority over optional extensions. Kafka and Airflow must remain separate from the core delivery path until the required components work.

A change must not be accepted if it compromises the four-week deadline without explicit agreement from the project supervisor or team.

Weekly Reviews

The team will review progress during each project week.

Reviews will confirm completed deliverables, test results, technical blockers, and changes that affect scope or implementation priorities.

Expected Deliverables

The requirements gathering process will produce the following project artifacts:

- Agreed project scope and data-flow design.
- Selected dataset and documented Data Dictionary.
- Python ingestion, cleaning, and validation pipeline.
- Microsoft SQL Server schema and analytical queries.
- HDFS storage, PySpark transformations, and Parquet outputs, subject to the approved Big Data scope.
- Streamlit monitoring dashboard.
- Test cases, reconciliation results, and data-quality reports.
- Setup instructions and technical documentation.
- Requirements traceability tracker and final demonstration.